Wykład 4:

1 / 83

Wykład 4: Wstęp JavaScript

1 / 83

Plan wykładu

1. Wstęp do JavaScript
2. Cechy języka
3. Zmienne i wyrażenia
4. Funkcje
5. Zasięg zmiennych
6. Instrukcje kontrolne
7. Obiekty
8. Tablice
9. Frameworki
10. jQuery

2 / 83

Historia JavaScript

JavaScript to język programowania zaprojektowany przez firmę Netscape Communications dla przeglądarki internetowej.

3 / 83

Historia JavaScript

JavaScript to język programowania zaprojektowany przez firmę Netscape Communications dla przeglądarki internetowej.

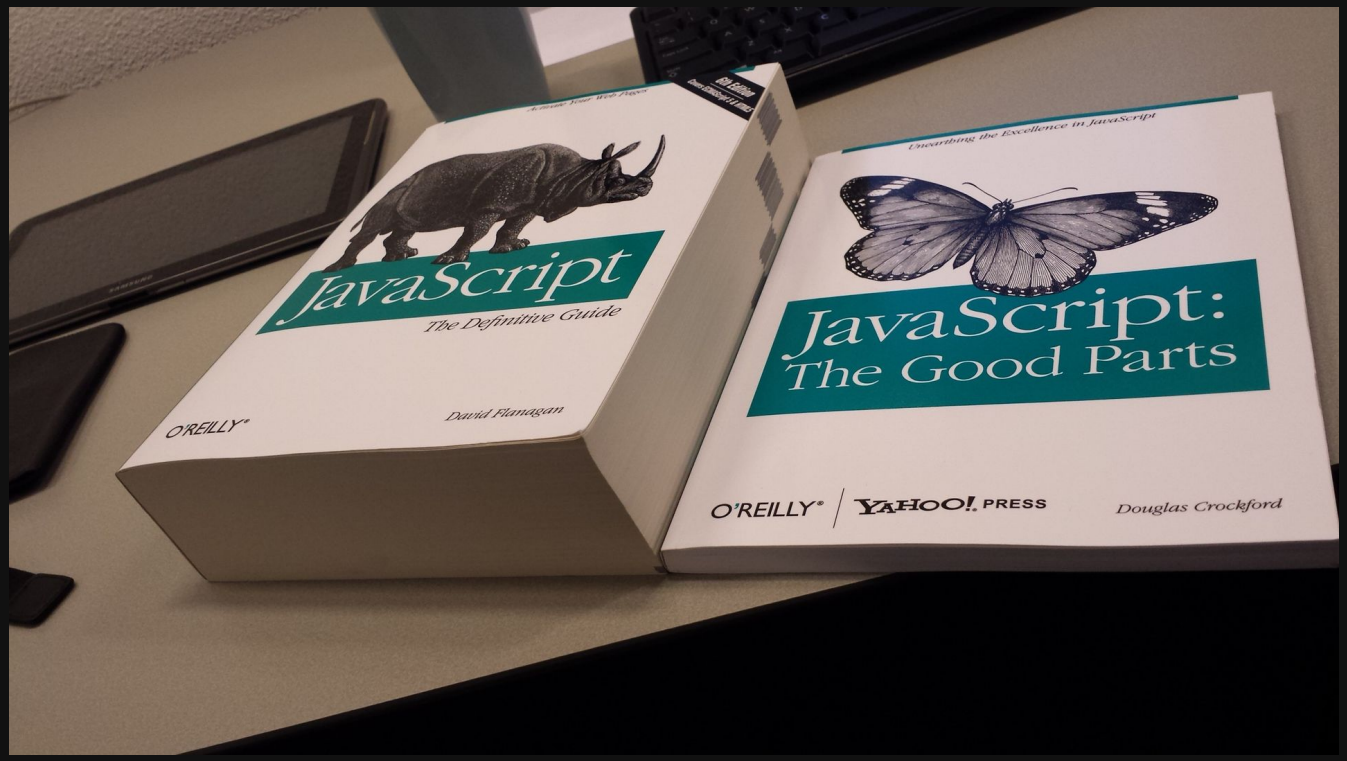
Twórcą języka jest Brendan Eich: *"To defend the idea of JavaScript against competing proposals, the company needed a prototype. Eich wrote one in 10 days, in May 1995."*

3 / 83

JavaScript: The Good Parts, Douglas Crockford

4 / 83

JavaScript: The Good Parts, Douglas Crockford



4 / 83

JavaScript łączy w sobie trzy koncepcje:

5 / 83

JavaScript łączy w sobie trzy koncepcje:

- funkcyjność wzorowaną na języku *Scheme*,

5 / 83

JavaScript łączy w sobie trzy koncepcje:

- funkcyjność wzorowaną na języku *Scheme*,
- obiektowość wzorowana na języku *Smalltalk*,

5 / 83

JavaScript łączy w sobie trzy koncepcje:

- funkcyjność wzorowaną na języku *Scheme*,
- obiektowość wzorowana na języku *Smalltalk*,
- składnię wzorowaną na języku *Java*.

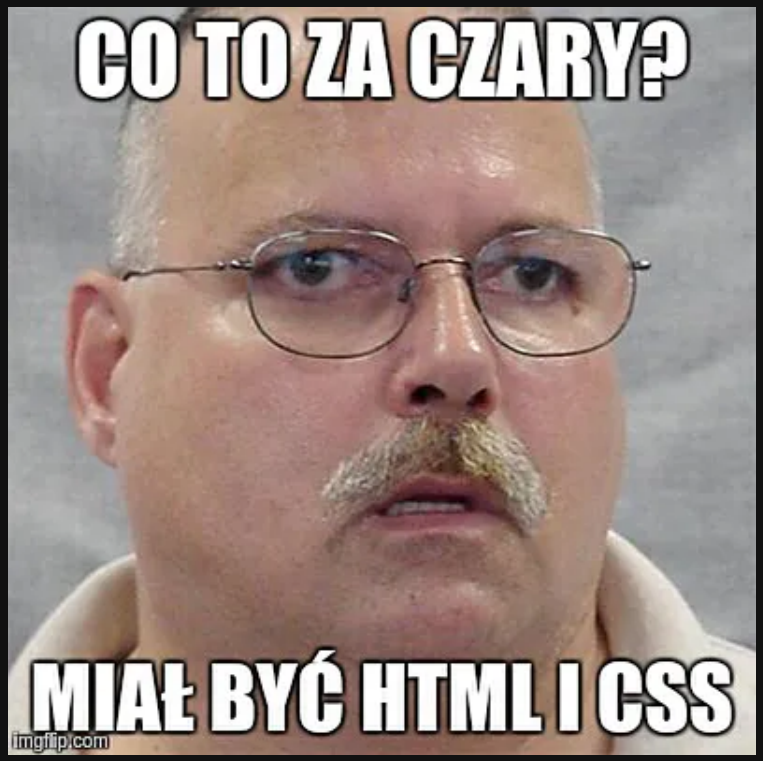
5 / 83

JavaScript łączy w sobie trzy koncepcje:

- funkcyjność wzorowaną na języku *Scheme*,
- obiektowość wzorowana na języku *Smalltalk*,
- składnię wzorowaną na języku *Java*.

Język JavaScript nie jest wariantem języka Java!!!

5 / 83


CO TO ZA CZARY?
MIAŁ BYĆ HTML I CSS
imgflip.com

6 / 83

Cechy języka

7 / 83

Cechy języka

- *Przenośność* - identyczne działanie niezależnie od platformy sprzętowej.

7 / 83

Cechy języka

- *Przenośność* - identyczne działanie niezależnie od platformy sprzętowej.
- *Dynamiczna typizacja* - typ nie jest ściśle przypisany do konkretnej zmiennej lub wyrażenia.

7 / 83

Cechy języka

- *Przenośność* - identyczne działanie niezależnie od platformy sprzętowej.
- *Dynamiczna typizacja* - typ nie jest ściśle przypisany do konkretnej zmiennej lub wyrażenia.
- *Jednowątkowość*

7 / 83

Dlaczego JavaScript jest ważny?

- dostępny wszędzie
- każdy użytkownik z reguły posiada przeglądarkę
- automatyczna aktualizacja

8 / 83

Najprostszy "przypadek użycia"

Osadzenie w stronie HTML

```
<html>
  <body>
    <script>
      console.log('Hello world');
    </script>
  </body>
</html>
```

9 / 83

Użycie osobnego pliku źródłowego

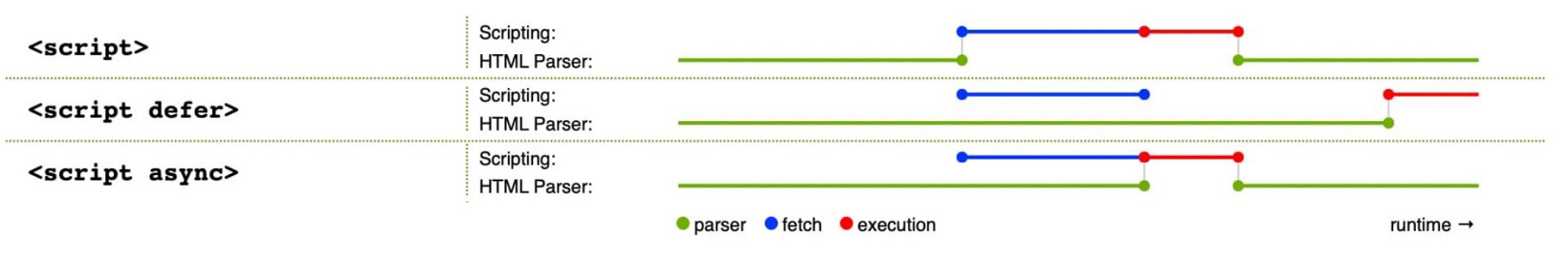
index.html

```
<html>
  <body>
    <script src="script.js"/>
  </body>
</html>
```

script.js

```
console.log('Hello world');
```

10 / 83


<script>
Scripting:
HTML Parser:
<script defer>
Scripting:
HTML Parser:
<script async>
Scripting:
HTML Parser:
parser
fetch
execution
runtime →

11 / 83

Wybrane frameworki

- Angular - tworzenie aplikacji SPA
- node.js - tworzenie aplikacji serwerowych
- Vue.js - tworzenie interfejsu użytkownika
- React.js - tworzenie interfejsu użytkownika
- Express.js - tworzenie aplikacji

12 / 83

Zmienne i stałe

Zmienne deklarujemy przy użyciu słowa kluczowego `var`:

```
var a = 10, b;   // a -> 10, b -> undefined
var c = null;    // c -> null
```

lub `let`:

```
let a = 10, b;   // a -> 10, b -> undefined
let c = null;    // c -> null
```

natomiast stałe za pomocą `const`:

```
const f = 20;    // f -> 20
```

13 / 83

14 / 83

- Niezainicjowania zmienna przyjmuje wartość `undefined`.

14 / 83

- Niezainicjowania zmienna przyjmuje wartość `undefined`.
- Słowo kluczowe `var`|`let` jest opcjonalne, zaleca się jednak je stosować.

14 / 83

Typy danych

- **JavaScript jest językiem dynamicznie typizowanym** - typ zmiennej lub wyrażenia nie jest znany podczas kompilacji.
- Podczas wykonywania kodu typ wartości przechowywanej w zmiennej lub będącej wynikiem wyrażenia jest dobrze określony.

15 / 83

Typy danych

- **JavaScript jest językiem dynamicznie typizowanym** - typ zmiennej lub wyrażenia nie jest znany podczas kompilacji.
- Podczas wykonywania kodu typ wartości przechowywanej w zmiennej lub będącej wynikiem wyrażenia jest dobrze określony.

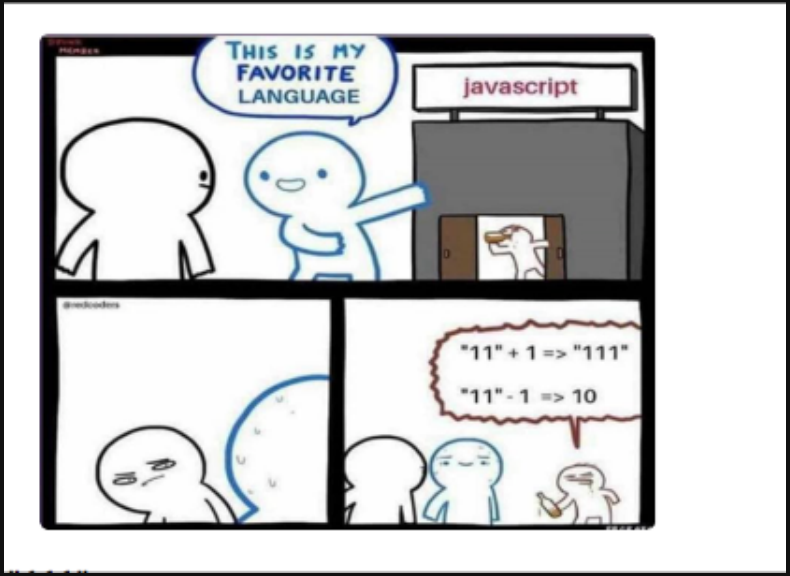
```
1 var a;
2 a = 2 + 2;
3 console.log(typeof(a)); // -> number
4 a = 'foo';
5 console.log(typeof(a)); // -> string
```

15 / 83

Typy danych

typ	przykłady
number	0, 1, 0.5, 2E-10
boolean	true, false
string	'', 'foo'
function	function(a) { return a + 1; }
array	[], ['apple', 'orange']
object	{}, {id: 5, value: 'foo'}

16 / 83


THIS IS MY FAVORITE LANGUAGE
javascript
"11" + 1 => "111"
"11" - 1 => 10

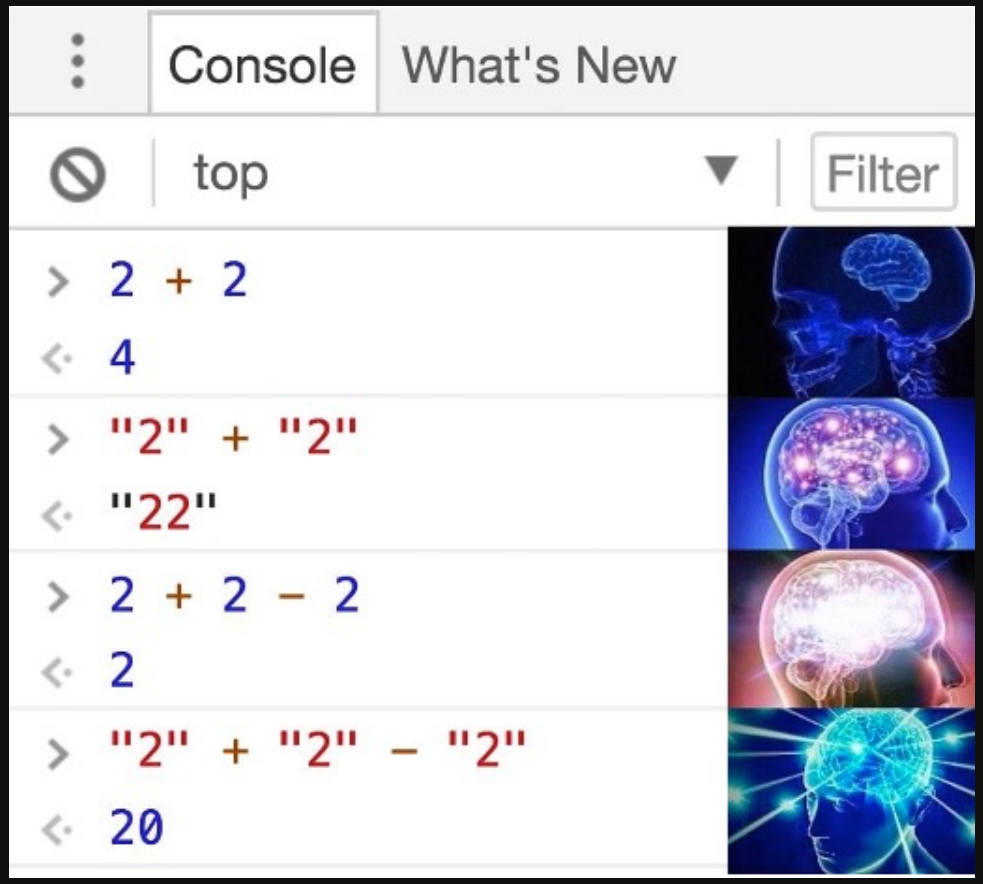
17 / 83

```
> typeof NaN
"number"
> 9999999999999999
10000000000000000
> 0.5+0.1==0.6
true
> 0.1+0.2==0.3
false
> Math.max()
-Infinity
> Math.min()
Infinity
> []+[]
""
> []+{}
"[object Object]"
> {}+[]
0
> true+true+true===3
true
> true-true
```

```
> true==1
true
> true===1
false
> (!+[]+[]+![]).length
9
> 9+"1"
"91"
> 91-"1"
90
> []==0
true
```

0

18 / 83


Console
What's New
top
Filter
2 + 2
4
"2" + "2"
"22"
2 + 2 - 2
2
"2" + "2" - "2"
20

19 / 83

Funkcje

Funkcja jest wartością utworzoną przy użyciu literału funkcyjnego:

```
function add(left, right) {
  return left + right;
};
add(2, 2); // -> 4
```

20 / 83

a co jeśli...

```
1 function add(left, right) {
2   return
3   left + right;
4 };
5 add(2, 2); // ?
```

```
1 function add(left, right) {
2   left + right;
3 };
4 add(2, 2); // ?
```

21 / 83

funkcja to też typ danych...

```
1  function map(array, fn) {
2    var result = [];
3    for(i = 0; i < array.length; ++i) {
4      result[i] = fn ? fn(array[i]) : array[i];
5    }
6    return result;
7  };
8
9  const array = [1,2,3,4,5];
10 const res1 = map(array);
11 const incrementFn = function(a) { return a + 1; };
12
13 map(array, incrementFn);
14 map(array, function(a) { return a * 2; });
15 map(array, incrementFn()); // ?
```

22 / 83

wartością funkcji może być funkcja

```
1  function createMultiplier(amount) {
2    return function(a) { return a * amount; };
3  };
4  createMultiplier(5)(2); // -> 10
5
6  function map(array, fn) {
7    var result = [];
8    for(i = 0; i < array.length; ++i) {
9      result[i] = fn ? fn(array[i]) : array[i];
10   }
11   return result;
12 };
```

23 / 83

Na szczęście funkcja map jest standardowo zdefiniowana dla typu tablicowego:

```
[1,2,3,4,5].map(createMultiplier(2));
// -> [2,4,5,8,10];
```

24 / 83

W języku JavaScript funkcję można zadeklarować (stworzyć) na kilka sposobów...

```
1  var add = function(left, right) {
2    return left + right;
3  };
4  function sub(left, right) {
5    return left - right;
6  };
7  const mul = (left, right) => left * right;
8
9  // w konsekwencji...
10 add(2, 2); // -> 4
11 function add(left, right) {
12   return 0;
13 }
14 add(2, 2); // -> 0
15 add = null;
16 add(2, 2); // -> Uncaught TypeError: add is not a function
```

25 / 83

W języku JavaScript funkcję można zadeklarować (stworzyć) na kilka sposobów...

```
1  var add = function(left, right) {
2    return left + right;
3  };
4  function sub(left, right) {
5    return left - right;
6  };
7  const mul = (left, right) => left * right;
8
9  // w konsekwencji...
10 add(2, 2); // -> 4
11 function add(left, right) {
12   return 0;
13 }
14 add(2, 2); // -> 0
15 add = null;
16 add(2, 2); // -> Uncaught TypeError: add is not a function
```

25 / 83

W języku JavaScript funkcję można zadeklarować (stworzyć) na kilka sposobów...

```
1  var add = function(left, right) {
2    return left + right;
3  };
4  function sub(left, right) {
5    return left - right;
6  };
7  const mul = (left, right) => left * right;
8
9  // w konsekwencji...
10 add(2, 2); // -> 4
11 function add(left, right) {
12   return 0;
13 }
14 add(2, 2); // -> 0
15 add = null;
16 add(2, 2); // -> Uncaught TypeError: add is not a function
```

25 / 83

W języku JavaScript funkcję można zadeklarować (stworzyć) na kilka sposobów...

```
1  var add = function(left, right) {
2    return left + right;
3  };
4  function sub(left, right) {
5    return left - right;
6  };
7  const mul = (left, right) => left * right;
8
9  // w konsekwencji...
10 add(2, 2); // -> 4
11 function add(left, right) {
12   return 0;
13 }
14 add(2, 2); // -> 0
15 add = null;
16 add(2, 2); // -> Uncaught TypeError: add is not a function
```

25 / 83

W języku JavaScript funkcję można zadeklarować (stworzyć) na kilka sposobów...

```
1  var add = function(left, right) {
2    return left + right;
3  };
4  function sub(left, right) {
5    return left - right;
6  };
7  const mul = (left, right) => left * right;
8
9  // w konsekwencji...
10 add(2, 2); // -> 4
11 function add(left, right) {
12   return 0;
13 }
14 add(2, 2); // -> 0
15 add = null;
16 add(2, 2); // -> Uncaught TypeError: add is not a function
```

25 / 83

Operatory w JavaScript

- , [], ()
- Arytmetyczne: +, -, *, /, %, ++, --, ** (ES7)
- Konkatenacja łańcuchów znaków: +, +=
- Bitowe: &, |, ^, ~, <<, >>, >>>
- Przypisania: =, +=, -=, ..., <<=, ..., &=, ...
- Porównania: ==, !=, ===, !==, >, >=, <, <=
- Logiczne: &&, ||, ! (skrócone wartościowanie && i ||)
- new, delete, typeof, void, in, instanceof
- Operator przecinka: ,
- Operator warunkowy (ternarny): ?:
- Destrukturyzacja tablicy/obiektu (ES6)
- Operatory rest i spread: ... (ES6)

26 / 83

Zmienne globalne

Użycie zmiennej wewnątrz ciała funkcji bez wcześniejszej deklaracji z użyciem słowa *var* spowoduje użycie lub stworzenie zmiennej globalnej.

27 / 83

Zmienne globalne

Użycie zmiennej wewnątrz ciała funkcji bez wcześniejszej deklaracji z użyciem słowa *var* spowoduje użycie lub stworzenie zmiennej globalnej.

Jest to fatalny błąd projektowy, który popełnili twórcy języka JavaScript!

27 / 83

Zmienne globalne

Użycie zmiennej wewnątrz ciała funkcji bez wcześniejszej deklaracji z użyciem słowa *var* spowoduje użycie lub stworzenie zmiennej globalnej.

Jest to fatalny błąd projektowy, który popełnili twórcy języka JavaScript!

```
var a = 10;
var f = function() {
  console.log(a); // -> 10 !
  a = 99;
};
f();
console.log(a); // -> 99 !!!
```

27 / 83

Zmienne globalne

Użycie zmiennej wewnątrz ciała funkcji bez wcześniejszej deklaracji z użyciem słowa *var* spowoduje użycie lub stworzenie zmiennej globalnej.

Jest to fatalny błąd projektowy, który popełnili twórcy języka JavaScript!

```
var a = 10;
var f = function() {
  console.log(a); // -> 10 !
  a = 99;
};
f();
console.log(a); // -> 99 !!!
```

```
var f = function() {
  a = 99;
};
f();
console.log(a); // -> 99 !!!
```

27 / 83

Zmienne globalne

Szczególnie groźne jest błędne użycie zmiennej indeksującej w pętli *for*:

```
1  function processRow(row) {
2    for (i = 0; i < row.length; ++i) {
3      console.log(row[i]);
4    }
5  }
6  function processMatrix(matrix) {
7    for (i = 0; i < matrix.length; ++i) {
8      processRow(matrix[i]);
9    }
10 };
11 processMatrix([[1,2,3], [4,5,6], [7,8,9]]);
```

28 / 83

Zmienne globalne

Szczególnie groźne jest błędne użycie zmiennej indeksującej w pętli *for*:

```
1  function processRow(row) {
2    for (i = 0; i < row.length; ++i) {
3      console.log(row[i]);
4    }
5  }
6  function processMatrix(matrix) {
7    for (i = 0; i < matrix.length; ++i) {
8      processRow(matrix[i]);
9    }
10 };
11 processMatrix([[1,2,3], [4,5,6], [7,8,9]]);
```

-> 1 2 3

28 / 83

Zmienne globalne

```
1  function processRow(row) {
2    for (var i = 0; i < row.length; ++i) {
3      console.log(row[i]);
4    }
5  }
6  function processMatrix(matrix) {
7    for (var i = 0; i < matrix.length; ++i) {
8      processRow(matrix[i]);
9    }
10 };
11 processMatrix([[1,2,3], [4,5,6], [7,8,9]]);
```

29 / 83

Zmienne globalne

```
1  function processRow(row) {
2    for (var i = 0; i < row.length; ++i) {
3      console.log(row[i]);
4    }
5  }
6  function processMatrix(matrix) {
7    for (var i = 0; i < matrix.length; ++i) {
8      processRow(matrix[i]);
9    }
10 };
11 processMatrix([[1,2,3], [4,5,6], [7,8,9]]);
```

-> 1 2 3 4 5 6 7 8 9

29 / 83

Zmienne lokalne

Wewnątrz funkcji zawsze deklarujemy zmienne z użyciem słowa *var* (*let*, *const*), gdyż wtedy stają się one zmiennymi **lokalnymi**.

30 / 83

Hoisting

Zasięg zmiennych w JavaScripcie jest **funkcyjny a nie blokowy**, chociaż blokowa składnia sugeruje coś innego.

31 / 83

Hoisting

Zasięg zmiennych w JavaScripcie jest **funkcyjny a nie blokowy**, chociaż blokowa składnia sugeruje coś innego.

```
1  function f1() {
2    console.log(a); // -> Uncaught ReferenceError: a is not defined
3  };
4  function f2() {
5    console.log(a); // -> undefined
6    var a = 10;
7    console.log(a); // -> 10
8  };
9  function f3() {
10   console.log(a); // -> undefined
11   { // <- to nie ma znaczenia dla widzialności
12     var a = 10;
13     console.log(a); // -> 10
14   }
15 };
```

31 / 83

Hoisting

Zasięg zmiennych w JavaScripcie jest **funkcyjny a nie blokowy**, chociaż blokowa składnia sugeruje coś innego.

```
1  function f1() {
2    console.log(a); // -> Uncaught ReferenceError: a is not defined
3  };
4  function f2() {
5    console.log(a); // -> undefined
6    var a = 10;
7    console.log(a); // -> 10
8  };
9  function f3() {
10   console.log(a); // -> undefined
11   { // <- to nie ma znaczenia dla widzialności
12     var a = 10;
13     console.log(a); // -> 10
14   }
15 };
```

31 / 83

Hoisting

Zasięg zmiennych w JavaScripcie jest **funkcyjny a nie blokowy**, chociaż blokowa składnia sugeruje coś innego.

```
1  function f1() {
2    console.log(a); // -> Uncaught ReferenceError: a is not defined
3  };
4  function f2() {
5    console.log(a); // -> undefined
6    var a = 10;
7    console.log(a); // -> 10
8  };
9  function f3() {
10   console.log(a); // -> undefined
11   { // <- to nie ma znaczenia dla widzialności
12     var a = 10;
13     console.log(a); // -> 10
14   }
15 };
```

31 / 83

Domknięcia

```
1 const nextValue = function () {
2   let cntr = 0;
3   return function() {
4     ++cntr;
5     return cntr;
6   };
7 }();
8 console.log(nextValue()); // -> 1
9 console.log(nextValue()); // -> 2
```

32 / 83

Domknięcia

```
1 const nextValue = function () {
2   let cntr = 0;
3   return function() {
4     ++cntr;
5     return cntr;
6   };
7 }();
8 console.log(nextValue()); // -> 1
9 console.log(nextValue()); // -> 2
```

- Funkcja ma dostęp do parametrów i zmiennych widocznych w miejscu, w którym owa funkcja została zadeklarowana.*
- Owe parametry i zmienne są dostępne nawet wówczas, gdy funkcja wewnętrzna ma dłuższy czas życia niż funkcja zewnętrzna.
- Mechanizm ten nazywamy domknięciem (ang. *closure*).

* Wyjątkiem są zmienne *this* oraz *arguments*.

32 / 83

Instrukcje kontrolne

33 / 83

Instrukcja warunkowa

```
if(a === 3) {
    ...
} else {
    ...
}
```

34 / 83

Instrukcja warunkowa

```
if(a === 3) {
   ...
} else {
   ...
}
```

- Składnia jest identyczna jak w języku C lub Javie.

34 / 83

Instrukcja warunkowa

```
if(a === 3) {
   ...
} else {
   ...
}
```

- Składnia jest identyczna jak w języku C lub Javie.
- Wyrażenie nie musi być typu *boolean*.

34 / 83

Instrukcja warunkowa

```
if(a === 3) {
   ...
} else {
   ...
}
```

- Składnia jest identyczna jak w języku C lub Javie.
- Wyrażenie nie musi być typu *boolean*.
- Dla każdego wyrażenia w instrukcji *if* JavaScript określa czy jest ono *truthy* czy też *falsy*.

34 / 83

Instrukcja warunkowa

```
if(a === 3) {
   ...
} else {
   ...
}
```

- Składnia jest identyczna jak w języku C lub Javie.
- Wyrażenie nie musi być typu *boolean*.
- Dla każdego wyrażenia w instrukcji *if* JavaScript określa czy jest ono *truthy* czy też *falsy*.
- Do porównywania tożsamościowego stosujemy wyłącznie operatory `===` oraz `!==`.

34 / 83

Wartości *falsy* i *truthy*

Następujące wartości są *falsy*:

- `false`
- `null`
- `undefined`
- Pusty napis: `''`
- Liczba `0`
- Liczba `NaN`

35 / 83

Wartości *falsy* i *truthy*

Następujące wartości są *falsy*:

- `false`
- `null`
- `undefined`
- Pusty napis: `''`
- Liczba `0`
- Liczba `NaN`

Pozostałe wartości są *truthy* wliczając w to `true`, napis `'false'` oraz dowolną wartość obiektową.

35 / 83

Instrukcje pętli

```
while (expression) {
   ...
};
do {
   ...
} while (expression);
for (let i = 0; i < size; ++i) {
   ...
};
```

36 / 83

Instrukcje pętli

```
while (expression) {
   ...
};
do {
   ...
} while (expression);
for (let i = 0; i < size; ++i) {
   ...
};
```

36 / 83

- Wyrażenie expression podobnie jak w przypadku

Instrukcje pętli

```
while (expression) {
   ...
};
do {
   ...
} while (expression);
for (let i = 0; i < size; ++i) {
   ...
};
```

36 / 83

- Wyrażenie expression podobnie jak w przypadku

Obiekty

JavaScript oferuje bardzo wygodną formę deklarowania obiektów przy użyciu tzw *literału obiektowego*:

37 / 83

JavaScript oferuje bardzo wygodną formę deklarowania obiektów przy użyciu tzw *literału obiektowego*:

```
1 const person = {
2    name: 'John',
3    surname: 'Doe',
4    age: 30
5 };
```

37 / 83

JavaScript oferuje bardzo wygodną formę deklarowania obiektów przy użyciu tzw *literału obiektowego*:

```
1 const person = {
2    name: 'John',
3    surname: 'Doe',
4    age: 30
5 };
```

```
1 person.name;                          // -> 'John'
2 person['age'];                        // -> 30
3 person.email;                         // -> undefined
4 person.email || ''                    // -> ''
5 person.address.street                 // -> TypeError
6 person.address && person.address.street // -> undefined
```

37 / 83

Obiekty są strukturami dynamicznymi, można dodawać i usuwać właściwości "w locie":

38 / 83

Obiekty są strukturami dynamicznymi, można dodawać i usuwać właściwości "w locie":

```
1 const item = {};              // obiekt "pusty"
2 item.name = 'ATmega1284';
3 item.price = 199.90;
4 console.log(item);          // {name: "ATmega1284", price: 199.9}
5
6 delete item.name;
7 console.log(item);          // {price: 199.9}
```

38 / 83

Możliwe jest wyliczenie wszystkich właściwości obiektu przy użyciu specjalnej składni `for in`.

39 / 83

Możliwe jest wyliczenie wszystkich właściwości obiektu przy użyciu specjalnej składni `for in`.

```
1 const obj = {
2    name: 'John',
3    age: 30,
4    sayHello: function() { console.log('Hello ' + this.name); }
5 };
6 for(let property in obj) {
7    console.log(property + ':' + typeof obj[property]);
8 };
```

39 / 83

Metody

Obiekty mogą posiadać właściwości typu funkcyjnego. Tak zdefiniowane funkcje nazywamy metodami.

40 / 83

Metody

Obiekty mogą posiadać właściwości typu funkcyjnego. Tak zdefiniowane funkcje nazywamy metodami.

```
1  const account = {
2    balance: 0,
3    debit: function(amount) {
4      if (amount < 0) {
5        throw new Error('Illegal amount');
6      }
7      if (this.balance < amount) {
8        throw new Error('Insufficient account balance');
9      }
10     this.balance -= amount;
11     return this.balance;
12   },
13   credit: function(amount) {
14     if (amount < 0) {
15       throw new Error('Illegal amount');
16     }
```

40 / 83

Metody

Obiekty mogą posiadać właściwości typu funkcyjnego. Tak zdefiniowane funkcje nazywamy metodami.

```
1  const account = {
2    balance: 0,
3    debit: function(amount) {
4      if (amount < 0) {
5        throw new Error('Illegal amount');
6      }
7      if (this.balance < amount) {
8        throw new Error('Insufficient account balance');
9      }
10     this.balance -= amount;
11     return this.balance;
12   },
13   credit: function(amount) {
14     if (amount < 0) {
15       throw new Error('Illegal amount');
16     }
```

40 / 83

Metody

Obiekty mogą posiadać właściwości typu funkcyjnego. Tak zdefiniowane funkcje nazywamy metodami.

```
1  const account = {
2    balance: 0,
3    debit: function(amount) {
4      if (amount < 0) {
5        throw new Error('Illegal amount');
6      }
7      if (this.balance < amount) {
8        throw new Error('Insufficient account balance');
9      }
10     this.balance -= amount;
11     return this.balance;
12   },
13   credit: function(amount) {
14     if (amount < 0) {
15       throw new Error('Illegal amount');
16     }
```

40 / 83

Metody

Obiekty mogą posiadać właściwości typu funkcyjnego. Tak zdefiniowane funkcje nazywamy metodami.

```
5         throw new Error('Illegal amount');
6       }
7       if (this.balance < amount) {
8         throw new Error('Insufficient account balance');
9       }
10      this.balance -= amount;
11      return this.balance;
12    },
13    credit: function(amount) {
14      if (amount < 0) {
15        throw new Error('Illegal amount');
16      }
17      this.balance += amount;
18      return this.balance;
19    }
20  };
```

40 / 83

Zagadka

```
const account = {
  balance: 199,
  printBalance: function() {
        console.log(this.balance);
  }
};
```

41 / 83

Zagadka

```
const account = {
  balance: 199,
  printBalance: function() {
        console.log(this.balance);
  }
};
```

-> 199

41 / 83

Zagadka

```
const account = {
  balance: 199,
  printBalance: function() {
        console.log(this.balance);
  }
};
```

-> 199

```
const account = {
  balance: 199,
  printBalance: function() {
        function inner() {
            console.log(this.balance)
        };
        inner();
  }
};
```

41 / 83

Zagadka

```
const account = {
  balance: 199,
  printBalance: function() {
        console.log(this.balance);
  }
};
```

-> 199

```
const account = {
  balance: 199,
  printBalance: function() {
        function inner() {
            console.log(this.balance)
        };
        inner();
  }
};
```

-> undefined

41 / 83

Zagadka

```
const account = {
  balance: 199,
  printBalance: function() {
        console.log(this.balance);
  }
};
```

-> 199

```
const account = {
  balance: 199,
  printBalance: function() {
        function inner() {
            console.log(this.balance)
        };
        inner();
  }
};
```

-> undefined

41 / 83

Bezklasowość

JavaScript do wersji ES5 włącznie był językiem obiektowym, w którym nie istniało pojęcie klasy ani konstruktora.

42 / 83

Bezklasowość

JavaScript do wersji ES5 włącznie był językiem obiektowym, w którym nie istniało pojęcie klasy ani konstruktora.

Ponieważ istnieje tylko jeden typ obiektowy (wszystkie obiekty są tego samego typu), pojęcie klasy nie ma sensu.

42 / 83

Bezklasowość

JavaScript do wersji ES5 włącznie był językiem obiektowym, w którym nie istniało pojęcie klasy ani konstruktora.

Ponieważ istnieje tylko jeden typ obiektowy (wszystkie obiekty są tego samego typu), pojęcie klasy nie ma sensu.

Klasy są jednak także użyteczne jako wzorce do tworzenia obiektów, JavaScript oferuje inne mechanizmy wzorców.

Tworzenie obiektów

43 / 83

Sposób 1: literał obiektowy

```
1 var employee = {
2   name: 'John',
3   surname: 'Doe',
4   age: 30,
5   sayHello: function() {
6     console.log('Hello ' + this.name + ' ' + this.surname);
7   }
8 };
9 employee.sayHello();
```

44 / 83

Sposób 1: literał obiektowy

```
1 var employee = {
2   name: 'John',
3   surname: 'Doe',
4   age: 30,
5   sayHello: function() {
6     console.log('Hello ' + this.name + ' ' + this.surname);
7   }
8 };
9 employee.sayHello();
```

44 / 83

Sposób 1: literał obiektowy

```
1 var employee = {
2   name: 'John',
3   surname: 'Doe',
4   age: 30,
5   sayHello: function() {
6     console.log('Hello ' + this.name + ' ' + this.surname);
7   }
8 };
9 employee.sayHello();
```

44 / 83

Sposób 1: literał obiektowy

```
1 var employee = {
2   name: 'John',
3   surname: 'Doe',
4   age: 30,
5   sayHello: function() {
6     console.log('Hello ' + this.name + ' ' + this.surname);
7   }
8 };
9 employee.sayHello();
```

44 / 83

Sposób 2: funkcja fabrykująca

```
function makeEmployee(name, surname, age) {
  return {
    name: name,
    surname: surname,
    age: age,
    sayHello: function() {
      console.log(`Hello ${this.name} ${this.surname}`);
    }
  };
};
makeEmployee('John', 'Doe', 30).sayHello();
```

45 / 83

...lub bardziej elegancko przy użyciu obiektu specyfikującego

```
function makeEmployee(spec) {
  return {
    name: spec.name,
    surname: spec.surname,
    age: spec.age,
    sayHello: function() {
      console.log(`Hello ${this.name} ${this.surname}`);
    }
  };
};
makeEmployee({name: 'John', surname: 'Doe', age: 30}).sayHello();
```

46 / 83

Sposób 3: dziedziczenie prototypiczne

```
1  const employee = {
2    name: '',
3    surname: '',
4    sayHello: function() {
5      console.log(`Hello ${this.name} ${this.surname}`);
6    }
7  };
8  const john = Object.create(employee);
9  john.name = 'John';
10 john.surname = 'Doe';
11 john.sayHello(); // -> Hello John Doe
```

47 / 83

Sposób 3: dziedziczenie prototypiczne

```
1  const employee = {
2    name: '',
3    surname: '',
4    sayHello: function() {
5      console.log(`Hello ${this.name} ${this.surname}`);
6    }
7  };
8  const john = Object.create(employee);
9  john.name = 'John';
10 john.surname = 'Doe';
11 john.sayHello(); // -> Hello John Doe
```

Każda zmiana w prototypie jest natychmiast widoczna w obiektach dziedziczących.

```
1  employee.sayHello = function() {
2    console.log('Hello ' + this.name);
```

```
3 };
4 john.sayHello(); // -> Hello John
```

47 / 83

Sposób 4: funkcja fabrykująca i pola prywatne

W obiektach JavaScriptowych nie ma pól (a więc i metod) prywatnych, ale można ukryć pola stosując technikę domknięć (*closure*).

```
1  function makeEmployee(spec) {
2    const name = spec.name;
3    const surname = spec.surname;
4    return {
5      sayHello: function() {
6        console.log(`Hello ${name} ${surname}`);
7      }
8    };
9  };
10 var john = makeEmployee({name: 'John', surname: 'Doe'});
11 john.sayHello(); // -> Hello John Doe
12 john.name; // undefined
13 john.surname; // undefined
```

48 / 83

Tablice

Tablice w języku JavaScript są obiektami o pewnej specjalnej charakterystyce:

```
1 const t = [];
2 t[0] = 123; // --> [123]
3 t[5] = 3; // --> [123, 4 empty slots, 5]
4 t[6] = 'foo'; // --> [123, 4 empty slots, 5, 'foo bar']
5 t.foo = 'bar'; // --> [123, 4 empty slots, 5, 'foo bar']
6 console.log(t.foo); // --> bar
```

49 / 83

Tablice w języku JavaScript są obiektami o pewnej specjalnej charakterystyce:

```
1 const t = [];
2 t[0] = 123; // --> [123]
3 t[5] = 3; // --> [123, 4 empty slots, 5]
4 t[6] = 'foo'; // --> [123, 4 empty slots, 5, 'foo bar']
5 t.foo = 'bar'; // --> [123, 4 empty slots, 5, 'foo bar']
6 console.log(t.foo); // --> bar
```

```
const fibb = [1,2,3,5,8,13];
```

49 / 83

Tablice w języku JavaScript są obiektami o pewnej specjalnej charakterystyce:

```
1 const t = [];
2 t[0] = 123; // --> [123]
3 t[5] = 3; // --> [123, 4 empty slots, 5]
4 t[6] = 'foo'; // --> [123, 4 empty slots, 5, 'foo bar']
5 t.foo = 'bar'; // --> [123, 4 empty slots, 5, 'foo bar']
6 console.log(t.foo); // --> bar
```

```
const fibb = [1,2,3,5,8,13];
```

```
const bag = [1,5,'foo',{}, {name: 'Doe'}, [1,2,3]];
```

49 / 83

Iterowanie klasyczne:

```
const fibb = [1,2,3,5,8,13];
for(var i = 0; i < fibb.length; ++i) {
  console.log(`${i}: ${fibb[i]}`);
};
```

Iterowanie funkcyjne:

```
const fibb = [1,2,3,5,8,13];
fibb.forEach(function(val, i) {
  console.log(`${i}: ${val}`);
});
```

50 / 83

DOM - Document Object Model

- Opis reprezentacji dokumentu HTML lub XML w postaci drzewa elementów
- Interfejs do manipulowania elementami dokumentu
- Zdarzenia:
  - generowane przez mysz
  - generowane przez klawiaturę
  - związane z ramkami i obiektami
  - związane z formularzami

51 / 83

JavaScript - obiekt document

- Reprezentuje dokument wyświetlony w przeglądarce
- Zawiera wszystkie elementy składowe dokumentu
- Umożliwia manipulację cechami elementów: zachowanie, wygląd
- Dostęp do elemenotów
  - przez formularz: document.dataForm.nazwisko
  - przez nazwę: document.getElementsById("nazwisko)
- Operacje na dokumentcie: open(), clear(), write(). close()

52 / 83

Biblioteka jQuery

53 / 83

Dołączanie pliku jQuery z serwera CDN

54 / 83

```
    <script src="http://ajax.aspnetcdn.com/ajax/jQuery/jquery-1.6.3.min.js"

    <script src="http://code.jquery.com/jquery-1.6.3.min.js"></script>

    <script src="http://ajax.googleapis.com/ajax/libs/jquery/1.6.3/jquery.m
js"></script>
```

55 / 83

```
<script >
    $(document).ready(function() {
       // to jest miejsce na nasz kod
    });
</script>
```

56 / 83

Pobieranie elementów stron

57 / 83

```
$('selektor')
$('#banner')
$('.banner')
$('#navBar a')
```

58 / 83

```
$('#banner').html('<h1>Byłem tu, JavaScript</h1>');
```

59 / 83

Filtry

60 / 83

```
$('tr:even')
$('p:first')
$('p:last')
$('li:has(a)')
$('div:hidden').show()
```

61 / 83

Dodawanie treści do stron

62 / 83

```
.html()
.text()
.append()
.prepend()
.before()
.after()
```

63 / 83

Zastępowanie i usuwanie wybranych elementów

64 / 83

```
.remove()
.replace()
```

65 / 83

Ustawianie i odczyt atrybutów znaczników

66 / 83

```
.addClass()
.removeClass()
.toggleClass()
```

67 / 83

Odczyt i modyfikacja właściwości CSS

68 / 83

```
.css()
var bgColor = $('#main').css('background-color');
$('body').css('font-size', '200%');
$('#highlightedDiv').css({
  'background-color' : '#FF0000',
  'border' : '2px solid #FE0037'
});
```

69 / 83

Wykonanie akcji na każdym elemencie kolekcji

70 / 83

Funkcje anonimowe

71 / 83

Funkcje anonimowe

```
function() {
  // tu jest umieszczany kod funkcji
}
```

71 / 83

each()

72 / 83

each()

```
$('selektor').each(function(){
    // tu jest umieszczony kod funkcji
});
```

72 / 83

$(this)

73 / 83

$(this)

```
    var i=1;
    $('p').each(function(){
  var tekst = $(this).text();
    tekst=tekst+i;
    i++;
  $(this).text(tekst)
});
```

73 / 83

Akcja i reakcja

74 / 83

Mysz

75 / 83

Mysz

```
.click()
```

75 / 83

Mysz

```
$("p").click(function(){
    $(this).hide();
});
```

75 / 83

Mysz

76 / 83

Mysz

```
$("p").dblclick(function(){
    $(this).hide();
});
```

76 / 83

Mysz

77 / 83

Mysz

```
$("#p1").mouseenter(function(){
    alert("You entered p1!");
});
```

77 / 83

Mysz

78 / 83

Mysz

```
$("#p1").hover(function(){
alert("You entered p1!");
},
function(){
    alert("Bye! You now leave p1!");
});
```

78 / 83

Mysz

79 / 83

Mysz

```
$("p").on({
    mouseenter: function(){
        $(this).css("background-color", "lightgray");
    },
    mouseleave: function(){
        $(this).css("background-color", "lightblue");
    },
    click: function(){
        $(this).css("background-color", "yellow");
    }
});
```

79 / 83

Animacje i efekty

80 / 83

```
.hide()
.show()
.toggle()
.fadeOut()
.fadeIn()
.fadeToggle()
```

81 / 83

```
.slideDown()
.slideUp()
.slideToggle()
```

82 / 83

Źródła

- Politechnika Wrocławska - Projekt i implementacja systemó
- dr Piotr Rzonsowski - Uniwersytet im. Adama Mickiewicza w
- https://www.w3schools.com/jquery/
- https://www.w3schools.com/js/DEFAULT.asp
- https://www.tutorialspoint.com/javascript/index.htm
- https://developer.mozilla.org/pl/docs/Learn/Getting_starte

83 / 83